

1 Background

Floating Point (F): A number = mantissa × base^{exponent} (e.g. 2048 = 1 × 2¹¹ mantissa=1, base=2, exponent=11)

Assumptions: There is $\varepsilon_m > 0$ (machine epsilon) $A1: \forall \alpha \in \mathbb{R}, \exists \varepsilon \in (-\varepsilon_m, \varepsilon_m)$ s.t. $fl: \mathbb{R} \rightarrow \mathbb{F}$ by $fl(\alpha) = \alpha(1 + \varepsilon)$
 $\varepsilon_m \approx 2.22 \times 10^{-16}$ $A2: \forall \alpha, \beta \in \mathbb{F}, \forall * \in \{+, -, \times, \div\}, \exists \varepsilon \in (-\varepsilon_m, \varepsilon_m)$ s.t. $\alpha \otimes \beta = (\alpha * \beta)(1 + \varepsilon)$
 $A1$: The rounding error in α is relative to the size of α ($|\alpha - fl(\alpha)| < |\alpha|\varepsilon_m$) $A2$: The error in $\alpha \otimes \beta$ is relative to the size of $\alpha * \beta$ ($|\alpha \otimes \beta - \alpha * \beta| \leq |\alpha * \beta|\varepsilon_m$)

Cost of Algorithm (C): $C :=$ number of floating point operations (FLOPs) If $C(n) \leq \alpha f(n), \Rightarrow C(n) = \mathcal{O}(f(n))$.

We consider that: 交换/赋值/相反数 no cost (i.e. Permutation P is free), 比较两数 1 FLOPs, $\sqrt{\quad}$ 1 FLOPs.

Norms: $\|\mathbf{x}\|_p := (\sum_{i=1}^n |x_i|^p)^{1/p}, p \geq 1$ $\|\mathbf{x}\|_\infty := \max_{1 \leq i \leq n} |x_i|$ $\|\mathbf{x}\|_2 := (\sum_{i=1}^n |x_i|^2)^{1/2}$ $\|\mathbf{x}\|_1 := \sum_{i=1}^n |x_i|$

- 1. $\|\mathbf{x}\|_p \geq 0$ $\|\mathbf{x}\|_p = 0 \Leftrightarrow \mathbf{x} = \mathbf{0}$
- 2. $\|\alpha \mathbf{x}\|_p = |\alpha| \|\mathbf{x}\|_p$ for $\alpha \in \mathbb{R}$
- 3. $\|\mathbf{x} + \mathbf{y}\|_p \leq \|\mathbf{x}\|_p + \|\mathbf{y}\|_p$ (Triangle inequality)
- 4. Induced Norm: $\|\mathbf{A}\|_p := \sup_{\mathbf{x} \neq \mathbf{0}} \frac{\|\mathbf{Ax}\|_p}{\|\mathbf{x}\|_p} = \sup_{\|\mathbf{x}\|_p=1} \|\mathbf{Ax}\|_p$

1. $\|\mathbf{A}_{m \times n}\|_\infty := \max_{1 \leq j \leq m} \sum_{i=1}^n |a_{ij}|$ (maximum row sum)

2. $\|\mathbf{A}_{m \times n}\|_1 := \max_{1 \leq i \leq n} \sum_{j=1}^m |a_{ij}|$ (maximum column sum)

3. $\|\mathbf{A}_{m \times n}\|_2 := \sqrt{\rho(\mathbf{A}^T \mathbf{A})}$ $\rho(\mathbf{A}^T \mathbf{A}) = \max\{|\lambda| : \lambda \text{ is eigenvalue of } \mathbf{A}^T \mathbf{A}\}$

For Matrix: $\|\mathbf{Ax}\|_p \leq \|\mathbf{A}\|_p \|\mathbf{x}\|_p$ $\|\mathbf{AB}\|_p \leq \|\mathbf{A}\|_p \|\mathbf{B}\|_p$ $\|\mathbf{A}^k\|_p \leq \|\mathbf{A}\|_p^k$ $\|\mathbf{A} + \mathbf{B}\|_p \leq \|\mathbf{A}\|_p + \|\mathbf{B}\|_p$

2 Methods for Systems of Linear Equations (SLE)

Ideal: We want to solve SLE: $\mathbf{Ax} = \mathbf{b}$, where $\mathbf{A}_{n \times n}$ is invertible, $\mathbf{x}, \mathbf{b} \in \mathbb{R}^n$

2.1 Backward Stable | Condition Number | Error

Backward Stable: Algorithm solve SLE is Backward Stable if: $\exists \alpha(n) > 0$ s.t. $\forall \mathbf{A}, \mathbf{b}$, computed solution $\hat{\mathbf{x}}$ has: $(\mathbf{A} + \Delta \mathbf{A})\hat{\mathbf{x}} = \mathbf{b} + \Delta \mathbf{b}$ and $\frac{\|\Delta \mathbf{A}\|_p}{\|\mathbf{A}\|_p} + \frac{\|\Delta \mathbf{b}\|_p}{\|\mathbf{b}\|_p} \leq \alpha \varepsilon_m$ ε_m : machine epsilon.

Condition Number ($\kappa_p(\mathbf{A})$): $\kappa_p(\mathbf{A}) := \begin{cases} \frac{\|\mathbf{A}\|_p \|\mathbf{A}^{-1}\|_p}{1} \geq 1 & \text{if } \mathbf{A} \text{ is invertible} \\ \infty & \text{otherwise} \end{cases}$ If $\kappa_p(\mathbf{A})$ is large (e.g. $\kappa_p(\mathbf{A}) \geq 10^{12} \approx \varepsilon_m^{-1}$), $\mathbf{A}$ is ill-conditioned. If $\kappa_p(\mathbf{A}) \leq 10^4$, $\mathbf{A}$ is well-conditioned. ps: $\kappa_p(\mathbf{A})$ 衡量矩阵奇异性

If $\mathbf{A}_{n \times n}$ s.t. $|\lambda_1| \geq |\lambda_2| \geq \dots \geq |\lambda_n|$ corresponding $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$ are orthonormal set. (e.g. $\bar{\mathbf{A}}^T = \mathbf{A}$) Then $\kappa_2(\mathbf{A}) = \|\mathbf{A}\|_2 \|\mathbf{A}^{-1}\|_2 = \frac{|\lambda_1|}{|\lambda_n|} \geq 1$

Relative Error: $\frac{\|\hat{\mathbf{x}} - \mathbf{x}\|_p}{\|\mathbf{x}\|_p}$ Actual Error: $\|\hat{\mathbf{x}} - \mathbf{x}\|_p$ Let $\mathbf{Ax} = \mathbf{b}$ and $(\mathbf{A} + \Delta \mathbf{A})\hat{\mathbf{x}} = \mathbf{b}$. If $\kappa_p(\mathbf{A}) \frac{\|\Delta \mathbf{A}\|_p}{\|\mathbf{A}\|_p} < 1$, then: $\frac{\|\mathbf{x} - \hat{\mathbf{x}}\|_p}{\|\mathbf{x}\|_p} \leq \frac{\kappa_p(\mathbf{A})}{1 - \kappa_p(\mathbf{A}) \frac{\|\Delta \mathbf{A}\|_p}{\|\mathbf{A}\|_p}} \cdot \frac{\|\Delta \mathbf{A}\|_p}{\|\mathbf{A}\|_p}$ ps: $\frac{\|\Delta \mathbf{A}\|_p}{\|\mathbf{A}\|_p}$ 小代表 good stability.

Generally, Let $\mathbf{Ax} = \mathbf{b}$ and $(\mathbf{A} + \Delta \mathbf{A})\hat{\mathbf{x}} = \mathbf{b} + \Delta \mathbf{b}$. If $\kappa_p(\mathbf{A}) \frac{\|\Delta \mathbf{A}\|_p}{\|\mathbf{A}\|_p} < 1$ then $\frac{\|\mathbf{x} - \hat{\mathbf{x}}\|_p}{\|\mathbf{x}\|_p} \leq \frac{\kappa_p(\mathbf{A})}{1 - \kappa_p(\mathbf{A}) \frac{\|\Delta \mathbf{A}\|_p}{\|\mathbf{A}\|_p}} \cdot (\frac{\|\Delta \mathbf{A}\|_p}{\|\mathbf{A}\|_p} + \frac{\|\Delta \mathbf{b}\|_p}{\|\mathbf{b}\|_p})$ ps: $\frac{\|\Delta \mathbf{A}\|_p}{\|\mathbf{A}\|_p}$ $\frac{\|\Delta \mathbf{b}\|_p}{\|\mathbf{b}\|_p}$ 小代表 good stability.

2.2 LU Factorization | Gaussian Elimination (GE)

Def of LU: If $\mathbf{A}_{n \times n}$ is invertible, then $\exists \mathbf{L}, \mathbf{U}$: 单位下三角, $\exists \mathbf{U}$: 可逆上三角. s.t. $\mathbf{A} = \mathbf{LU}$ (unique)

LU Factorization: By Gaussian elimination, $\mathbf{A}_{n \times n} = (\prod_{k=1}^{n-1} \mathbf{L}_k)^{-1} \mathbf{U} := \mathbf{LU}$ $\mathbf{L}_k$: Lower triangular with 1s on the diagonal (by row operation on single column)

- 1. 如果 $\mathbf{L}$ 是单位下三角矩阵, 且只有第 k 列有非零元素在对角线下, 则 $\mathbf{L}^{-1}$ 矩阵是 $\mathbf{L}$ 对角线不动, 其余为相反数.
- 2. 如果 $\mathbf{L}_1, \mathbf{L}_2$ 矩阵是单位下三角矩阵, 且 $\mathbf{L}_1 \mathbf{L}_2$ 仅在第 $1 \sim k | k + 1 \sim n$ 列有非零元素在对角线下, 则 $\mathbf{L}_1 \mathbf{L}_2$ 矩阵是他们的"拼接".

Error Analysis of GE: Gaussian elimination is an unstable algorithm. (i.e. floating point arithmetic can have a disastrous effect on the computed solution.)

2.3 LUPP | LUCP | GEPP | GECP

Permutation Matrix: For a permutation π , $\mathbf{P}_\pi := [\mathbf{e}_{\pi(1)}, \dots, \mathbf{e}_{\pi(n)}]$ ps: 写的时候是对 i 第 i 行的 1 移动到第 $\pi(i)$ 行. 注意: 用另一方式定义的 $\mathbf{P}_\pi$ 会让后面到 π 结论不一样

- 左边乘 $\mathbf{P}_\pi$: 交换行 ($\mathbf{P}_\pi$ 的行 代表具体行的交换方法, π 也代表交换行的方法)
- 右边乘 $\mathbf{P}_\pi$: 交换列 ($\mathbf{P}_\pi$ 的列 代表具体列的交换方法, π^{-1} 才代表交换列的方法)
- Proposition: $\mathbf{P}_\pi$ is orthogonal, i.e. $\mathbf{P}_\pi^T = \mathbf{P}_\pi^{-1}$ · 对于二阶置换矩阵 (Involution): $\mathbf{P}_\pi^2 = \mathbf{I}$, i.e. $\mathbf{P}_\pi^{-1} = \mathbf{P}_\pi$ · GEPP|GECP: 交换行 (行和列) 使每次 $|u_{kk}|$ 最大 (stable)

Backward Stable of GEPP: $\forall$ SLE sol $\hat{\mathbf{x}}$ by GEPP, $\exists \Delta \mathbf{A}$ s.t. $(\mathbf{A} + \Delta \mathbf{A})\hat{\mathbf{x}} = \mathbf{b}$ and $\frac{\|\Delta \mathbf{A}\|_\infty}{\|\mathbf{A}\|_\infty} \leq p(n) 2^{n-1} \varepsilon_m$ $p(n)$ is a cubic polynomial

2.4 QR Factorization Method

QR Factorization: QR factorization of $\mathbf{A}_{n \times n}$ is $\mathbf{A} = \mathbf{QR}$, where $\mathbf{Q}$ is orthogonal (i.e. $\mathbf{Q}^T = \mathbf{Q}^{-1}$) and $\mathbf{R}$ is non-singular upper triangular.

- Def of $\mathbf{Q}$ is orthogonal is equivalent to $\mathbf{Q}^T \mathbf{Q} = \mathbf{I}$. (i.e. columns of $\mathbf{Q}$ forming an orthonormal set.) · QR produce large errors
- Orthogonal Projection: $\text{proj}_{\mathbf{u}}(\mathbf{x}) = \frac{\mathbf{x} \cdot \mathbf{u}}{\|\mathbf{u}\|_2^2} \mathbf{u} = (\mathbf{u}^T \mathbf{x}) \mathbf{u}$ if $\|\mathbf{u}\|_2 = 1$. $\mathbf{x} = \text{proj}_{\mathbf{u}}(\mathbf{x})_{\text{parallel to } \mathbf{u}} + (\mathbf{x} - \text{proj}_{\mathbf{u}}(\mathbf{x}))_{\text{orthogonal to } \mathbf{u}}$

Error Analysis of MGS: The matrices $\hat{\mathbf{Q}}$ and $\hat{\mathbf{Q}}$ computed by MGS satisfies: $\hat{\mathbf{Q}}^T \hat{\mathbf{Q}} = \mathbf{I} + \Delta \mathbf{I}$ $\hat{\mathbf{Q}} \hat{\mathbf{R}} = \mathbf{A} + \Delta \mathbf{A}$ where:

$\|\Delta \mathbf{I}\|_2 \leq \frac{\alpha_1(n) \varepsilon_m \kappa_2(\mathbf{A})}{1 - \alpha_1(n) \varepsilon_m \kappa_2(\mathbf{A})}$, if $1 - \alpha_1(n) \varepsilon_m \kappa_2(\mathbf{A}) > 0$ $\|\Delta \mathbf{A}\|_2 \leq \alpha_2(n) \varepsilon_m \|\mathbf{A}\|_2$ ps: $\alpha_1(n), \alpha_2(n)$ 只由 n 决定

2.5 Householder QR factorisation

Householder QR: (Most) stable algorithm Householder Reflection Matrix: $\mathbf{H}(\mathbf{w}) := \mathbf{I} - 2 \frac{\mathbf{w} \mathbf{w}^T}{\|\mathbf{w}\|_2^2}$ $\mathbf{H}(\mathbf{w}) \mathbf{x} = \mathbf{x} - 2 \text{proj}_{\mathbf{w}}(\mathbf{x})$

- $\mathbf{H}(\mathbf{w})$ is orthogonal and symmetric (In the Algorithm is $\mathbf{Q}_k, \mathbf{H}_k$) · $\mathbf{H}(\mathbf{w}) \mathbf{x}$ 作用为让 $\mathbf{x}$ 向量对 $\mathbf{w}$ 的法向量 (超平面) 上进行反射

Error Analysis of Householder QR: Let $\hat{\mathbf{x}}$ denote sol of $\mathbf{Ax} = \mathbf{b}$ by Householder QR+SQR. Then $(\mathbf{A} + \Delta \mathbf{A})\hat{\mathbf{x}} = \mathbf{b}$, where $\|\Delta \mathbf{A}\|_2 \leq \alpha n^{\frac{5}{2}} \varepsilon_m \|\mathbf{A}\|_2$ α small constant

2.6 Iterative Methods for SLE

Iterative Method: For SLE $\mathbf{Ax} = \mathbf{b}$, if $\mathbf{A} = \mathbf{M} + \mathbf{N}$ with $\mathbf{M}$ is simpler structure than $\mathbf{A}$. (例如: 对角/三角/对称矩阵) $\Rightarrow$ 迭代求接近似解 Diff choices $\mathbf{M}, \mathbf{N} \rightarrow$ different iterative methods.

Error Analysis: Error $\mathbf{e}_k := \mathbf{x} - \mathbf{x}_k \Rightarrow \mathbf{e}_k = \mathbf{R}^k \mathbf{e}_0$ $\mathbf{R} := -\mathbf{M}^{-1} \mathbf{N}$ Theorem: If $\|\mathbf{R}\|_p < 1$, then $\lim_{k \rightarrow \infty} \|\mathbf{e}_k\|_p = 0 \Leftrightarrow \lim_{k \rightarrow \infty} \mathbf{x}_k = \mathbf{x}$ (Convergence) GAU 收敛快于 JAC

JAC|GAU: If $\mathbf{A}$ is strictly diagonally dominant [i.e. $|a_{ii}| > \sum_{j \neq i} |a_{ij}| \forall i$ (对角线元素大于该行其他元素之和)], then converges for $p = \infty$. For $p = 1$ 看列和/转置.

3 Polynomial Fitting & LSQ

ps: 范德蒙矩阵 ill-cond

Vandermonde Matrix: $\mathbf{A} \in \mathbb{R}^{n \times m}$ s.t. $a_{ij} = x_i^{j-1}$ Poly: Deg $m - 1$ poly fit to points: $\{(a_i, b_i)\}_{i=1}^n$ by solving $\mathbf{Ax} = \mathbf{b} \Rightarrow$ sol $\mathbf{x}$ is coeff. of poly $p(x) = \sum_{j=1}^m x_j x^{j-1}$

LSQ: Finding line which best fits $\{(a_i, b_i)\}_{i=1}^n$ by minimise $\|\mathbf{Ax} - \mathbf{b}\|_2$, $\mathbf{A} \in \mathbb{R}^{n \times 2}$ vandermonde. Thm: minimises $\|\mathbf{Ax} - \mathbf{b}\|_2 \Leftrightarrow$ solve Normal Equations $\mathbf{A}^T \mathbf{Ax} = \mathbf{A}^T \mathbf{b}$

Moore-Penrose Pseudo-inverse of $\mathbf{A}$: $\mathbf{A}^\dagger = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T$ when $\text{rank}(\mathbf{A}) = m$, $(\mathbf{A}^T \mathbf{A})^{-1}$ exist · Normal Equations has unique sol if $\mathbf{A}^T \mathbf{A}$ invertible. · If $\mathbf{A}$ invertible, $\mathbf{A}^\dagger = \mathbf{A}^{-1}$

3.1 Singular Value Decomposition (SVD)

New Def of Condition Number: If $\mathbf{A}_{m \times n}$, $m \geq n$ s.t. it's full rank (i.e. $\text{rank}(\mathbf{A}) = n$), then $\kappa_p(\mathbf{A}) := \frac{\|\mathbf{A}\|_p \|\mathbf{A}^\dagger\|_p}{1} \geq 1$ $\kappa_2(\mathbf{A}^T \mathbf{A}) = \kappa_2(\mathbf{A})^2$ $\kappa_2(\mathbf{A}) = \frac{\sigma_{\max}}{\sigma_{\min}}$

SVD: $\mathbf{A}_{m \times n} = \mathbf{U} \Sigma \mathbf{V}^T$ where $\mathbf{U}_{m \times m}, \mathbf{V}_{n \times n}$ orthogonal, $\Sigma_{m \times n}$ diagonal with singular values $\sigma_1, \dots, \sigma_p, p = \min\{m, n\}$ $\mathbf{U}/\mathbf{V}$ 的列为 left/right singular vectors.

¹ Always exist ² $\mathbf{A} \mathbf{v}_i = \sigma_i \mathbf{u}_i$ and $\mathbf{A}^T \mathbf{u}_i = \sigma_i \mathbf{v}_i$ ³ $\sigma_i^2 |\mathbf{v}_i$ are eigenvalues/vectors of $\mathbf{A}^T \mathbf{A}$ (i.e. $\mathbf{A}^T \mathbf{A} \mathbf{v}_i = \sigma_i^2 \mathbf{v}_i$) ⁴ $\text{rank}(\mathbf{A}) = \text{rank}(\Sigma) = \#$ non-zero σ_i ⁵ $\mathbf{u}_i = \frac{1}{\sigma_i} \mathbf{A} \mathbf{v}_i$

Reduced QR Factorisation: By QR: $\mathbf{A}_{m \times n} = \mathbf{Q}_{m \times m} \mathbf{R}_{m \times n}, m \geq n$. ¹ $\mathbf{R}$ 的 n 行以下全 0 ² $\mathbf{Q} = [\mathbf{Q}_1 \mathbf{Q}_2], \mathbf{R} = [\mathbf{R}_1 \mathbf{0}]^T \Rightarrow \mathbf{A} = \mathbf{Q}_1 \mathbf{R}_1$ where $\mathbf{Q}_1, \mathbf{R}_1 \in \mathbb{R}^{n \times n}$

Reduced SVD: By SVD: $\mathbf{A} = \mathbf{U} \Sigma \mathbf{V}^T, m \geq n$. ¹ If $\mathbf{B}$ orthogonal, then $\mathbf{B}^{-1} = \mathbf{B}^T, \det(\mathbf{B}) = \pm 1$ and $\|\mathbf{Bx}\|_2 = \|\mathbf{x}\|_2, \forall \mathbf{x}$ ² $\mathbf{U} = [\mathbf{U}_1 \mathbf{U}_2], \Sigma = [\Sigma_1 \mathbf{0}]^T \Rightarrow \mathbf{A} = \mathbf{U}_1 \Sigma_1 \mathbf{V}^T, \mathbf{U}_1 \in \mathbb{R}^{m \times n}, \Sigma_1 \in \mathbb{R}^{n \times n}$

Error Analysis of LSQ-QR: Let $\hat{\mathbf{x}}$ denote sol of LSQ-QR (Householder). Then $\hat{\mathbf{x}}$ minimises $\|(\mathbf{A} + \Delta \mathbf{A})\mathbf{x} - \mathbf{b}\|_2$, and $\frac{\|\Delta \mathbf{A}\|_2}{\|\mathbf{A}\|_2} \leq \alpha n^{\frac{3}{2}} m \varepsilon_m, \alpha > 0$

Error Analysis LSQ (any): Let $\mathbf{x}|\hat{\mathbf{x}}$ minimises $\|\mathbf{Ax} - \mathbf{b}\|_2 \|(\mathbf{A} + \Delta \mathbf{A})\mathbf{x} - \mathbf{b}\|_2$ and $\text{rank}(\mathbf{A}) = \text{rank}(\mathbf{A} + \Delta \mathbf{A}) = n$. If $\kappa_2(\mathbf{A}) \frac{\|\Delta \mathbf{A}\|_2}{\|\mathbf{A}\|_2} < 1 \Rightarrow \frac{\|\mathbf{x} - \hat{\mathbf{x}}\|_2}{\|\mathbf{x}\|_2} \leq \frac{\kappa_2(\mathbf{A})}{1 - \kappa_2(\mathbf{A}) \frac{\|\Delta \mathbf{A}\|_2}{\|\mathbf{A}\|_2}} \cdot \frac{\|\Delta \mathbf{A}\|_2}{\|\mathbf{A}\|_2} \left(2 + (\kappa_2(\mathbf{A}) + 1) \frac{\|\mathbf{r}\|_2}{\|\mathbf{A}\|_2 \|\mathbf{x}\|_2} \right) \mathbf{r} = \mathbf{b} - \mathbf{Ax}$

4 Eigenvalue Problems

(In this section: $|\lambda_1| \geq |\lambda_2| \geq \dots \geq |\lambda_n|$ are eigenvalues of $\mathbf{A} \in \mathbb{R}^{n \times n}$ with corresponding eigenvectors $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$. $\lambda_1 = \lambda_{\max} \leftrightarrow \mathbf{x}_{\max}, \lambda_n = \lambda_{\min} \leftrightarrow \mathbf{x}_{\min}$)

Rayleigh Quotient: $r_{\mathbf{A}}(\mathbf{x}) = \frac{\mathbf{x}^T \mathbf{Ax}}{\mathbf{x}^T \mathbf{x}}$ $r_{\mathbf{A}}(\mathbf{x}_i) = \lambda_i$ Eigenvalues $\leftrightarrow$ Eigenvectors: $\mathbf{x}_i \rightarrow \lambda_i: r_{\mathbf{A}}(\mathbf{x}_i) = \lambda_i$ $\lambda_i \rightarrow \mathbf{x}_i: (\mathbf{A} - \lambda_i \mathbf{I})\mathbf{x}_i = \mathbf{0}$ Basis: $\mathbf{Ax} = \sum_i \alpha_i \lambda_i \mathbf{x}_i$

Convergence of Power Iteration: If $|\lambda_1| > |\lambda_2|$ and $\mathbf{x}_1 \cdot \mathbf{z}^{(0)} \neq 0$, then $\exists \sigma_{k \in \mathbb{N}}$ with $\sigma_k = \pm 1$ s.t. $\lim_{k \rightarrow \infty} \|\sigma_k \mathbf{z}^{(k)} - \mathbf{x}_1\|_2 = 0$ and $\lim_{k \rightarrow \infty} |\lambda^{(k)} - \lambda_1| = 0$

Useful Tool: Eigenvalues of $\mathbf{A}^{-1}$ are $\mu_i = \frac{1}{\lambda_i}$ and $\lambda_{\max} = \mu_{\min}^{-1}$ Eigenvalues of $(\mathbf{A} - s\mathbf{I})^{-1}$ are $\nu_i = \frac{1}{\lambda_i - s}$ and $\lambda_{\max} = s + \nu_{\min}^{-1}$

Inverse Iteration Algorithm: Input $\mathbf{A}^{-1}$ in Power Iteration $\Rightarrow$ find $\lambda_{\min}^{-1}$ and $\mathbf{x}_{\min}$ (用 SII $s = 0$ 可直接得到)

Shifted Inverse Iteration Algorithm: Using $(\mathbf{A} - s\mathbf{I})^{-1} \Rightarrow$ Find eigenvalue closest to s and corresponding eigenvector. Assume: $|\lambda_a - s| < |\lambda_b - s| < \dots$

Rate of Convergence for PI & SII: Rate $\uparrow$ iff $|\frac{\lambda_2}{\lambda_1}| \downarrow \|\sigma_k \mathbf{z}^{(k)} - \mathbf{x}_1\|_2 \leq \alpha_1 (\frac{|\lambda_2|}{|\lambda_1|})^k$; $|\lambda^{(k)} - \lambda_1| \leq \alpha_2 (\frac{|\lambda_2|}{|\lambda_1|})^{2k}$ Rate $\uparrow$ iff $|\frac{\lambda_a - s}{\lambda_b - s}| \downarrow \|\sigma_k \mathbf{z}^{(k)} - \mathbf{x}_i\|_2 \leq \beta_1 (\frac{|\lambda_a - s|}{|\lambda_b - s|})^k$; $|\lambda^{(k)} - \lambda_a| \leq \beta_2 (\frac{|\lambda_a - s|}{|\lambda_b - s|})^{2k}$

DP Dot product	$C = \sum_{i=1}^n 2 = 2n = \mathcal{O}(n)$
Input: $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ Output: $s = \mathbf{x} \cdot \mathbf{y} = \mathbf{x}^\top \mathbf{y}$ 1: $s = 0$ 2: for $i = 1, \dots, n$ do 3: $s = s + x_i y_i$ 4: end for	
Rank Theorem: $[\mathbf{A}\mathbf{x} = \mathbf{0} \text{ iff } \mathbf{x} = \mathbf{0}] \Leftrightarrow [\text{列线性无关}] \Leftrightarrow [\mathbf{A} \text{ 可逆}] \Leftrightarrow [\text{rank}(\mathbf{A}) = \text{rank}(\mathbf{A}^\top) = n]$ Abel: ≥ 5 次方 实系数多项式 3实根无法用有限次加减乘除和开方表示. Spectral Theorem: If $\mathbf{A}$ is symmetric, then $\exists$ orthonormal $\mathbf{Q}$ s.t. $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$ (Orthogonally diagonalisable) $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n)$	
LU LU factorisation	
$C(n) = \sum_{k=1}^{n-1} (n-k)(1+2(n-k+1)) = \frac{2}{3}n^3 + \frac{1}{2}n^2 - \frac{7}{6}n$	
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$ Output: $\mathbf{L}, \mathbf{U} \in \mathbb{R}^{n \times n}$ s.t. $\mathbf{A} = \mathbf{LU}$ 1: $\mathbf{L} = \mathbf{I}, \mathbf{U} = \mathbf{A}$ 2: for $k = 1, \dots, n-1$ do 3: for $j = k+1, \dots, n$ do 4: $l_{jk} = u_{jk}/u_{kk}$ 5: $(u_{jk}, \dots, u_{jn}) = (u_{jk}, \dots, u_{jn}) - l_{jk}(u_{kk}, \dots, u_{kn})$ 6: end for 7: end for	
GE Gaussian elimination	$C(n) = \frac{2}{3}n^3 + \frac{5}{2}n^2 - \frac{7}{6}n = \mathcal{O}(n^3)$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}, \mathbf{b} \in \mathbb{R}^n$ Output: $\mathbf{x} \in \mathbb{R}^n$ s.t. $\mathbf{Ax} = \mathbf{b}$ 1: Find LU factorisation of $\mathbf{A} = \mathbf{LU}$ # LU Algorithm 2: Solve $\mathbf{Ly} = \mathbf{b}$ using forward substitution. # FS Algorithm 3: Solve $\mathbf{Ux} = \mathbf{y}$ using back substitution. # BS Algorithm	

LUPP LU factorisation with partial pivoting (not unique)	$C(n) = \frac{2}{3}n^3 + n^2 - \frac{5}{3}n$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$ Output: $\mathbf{L}, \mathbf{U}, \mathbf{P} \in \mathbb{R}^{n \times n}$ s.t. $\mathbf{PA} = \mathbf{LU}$ with $\mathbf{L}$ 单位下三角, $\mathbf{U}$ 可逆上三角 1: $\mathbf{U} = \mathbf{A}, \mathbf{L} = \mathbf{I}, \mathbf{P} = \mathbf{I}$ 2: for $k = 1, \dots, n-1$ do 3: Choose $i \in \{k, \dots, n\}$ which maximizes $ u_{ik} $ # Max Algorithm 4: Exchange row $(u_{k1}, \dots, u_{kn})$ with $(u_{ik}, \dots, u_{in})$ # 第 k 行和第 i 行的 $k \sim n$ 列交换 5: Exchange row $(l_{k1}, \dots, l_{k,k-1})$ with $(l_{i1}, \dots, l_{i,k-1})$ # 第 k 行和第 i 行的 $1 \sim k-1$ 列交换 6: Exchange row $(p_{k1}, \dots, p_{kn})$ with $(p_{i1}, \dots, p_{in})$ # 第 k 行和第 i 行的全部列交换 7: for $j = k+1, \dots, n$ do 8: $l_{jk} = u_{jk}/u_{kk}$ 9: $(u_{jk}, \dots, u_{jn}) = (u_{jk}, \dots, u_{jn}) - l_{jk}(u_{kk}, \dots, u_{kn})$ 10: end for 11: end for	

(M)GS SQR Householder-QR

SQR (SLE) via QR-Factorization	$C(n) = 2n^3 + 4n^2 + n$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}, \mathbf{b} \in \mathbb{R}^n$ Output: $\mathbf{x} \in \mathbb{R}^n$ with $\mathbf{Ax} = \mathbf{b}$ 1: Find QR factorization of $\mathbf{A}$ # GS MGS Algorithm 2: Solve $\mathbf{Qy} = \mathbf{b}$ # MV Algorithm # ps: $\mathbf{y} = \mathbf{Q}^{-1}\mathbf{b} = \mathbf{Q}^\top \mathbf{b}$ 3: Solve $\mathbf{Rx} = \mathbf{y}$ using Algorithm BS # BS Algorithm	
Sum Notation: $\sum_{k=1}^n k = \frac{n(n+1)}{2} = \frac{1}{2}n^2 + \frac{1}{2}n$ $\sum_{k=1}^n k^2 = \frac{n(n+1)(2n+1)}{6} = \frac{1}{3}n^3 + \frac{1}{2}n^2 + \frac{1}{6}n$ $\sum_{k=1}^n k^3 = \left(\frac{n(n+1)}{2}\right)^2 = \frac{1}{4}n^4 + \frac{1}{2}n^3 + \frac{1}{4}n^2$	
Rank Theorem: $\mathbf{A}_{m \times n}, m \geq n: [\mathbf{Ax} = \mathbf{0} \text{ iff } \mathbf{x} = \mathbf{0}] \Leftrightarrow [\text{rank}(\mathbf{A}) = n]$ Cauchy-Schwarz Inequality: For $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n, \mathbf{x} \cdot \mathbf{y} = \mathbf{x}^\top \mathbf{y} \leq \ \mathbf{x}\ _2 \ \mathbf{y}\ _2$ Determinant: $\det(\mathbf{AB}) = \det(\mathbf{A})\det(\mathbf{B})$ Orthogonal: If $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$ Eigen: $\mathbf{A}\mathbf{v} = \lambda \mathbf{v}$ $\xi_{\mathbf{A}}(\lambda) = \det(\mathbf{A} - \lambda \mathbf{I}) = \prod_i (\lambda_i - \lambda)$	
Iterative Method OP MAX	
GI JAC GAU JAC: $C = \mathcal{O}(k_{\max} n^2)$ GAU: $C = \mathcal{O}(k_{\max} n^2)$	
Input: $\mathbf{A} = \mathbf{M} + \mathbf{N} \in \mathbb{R}^{n \times n}, \mathbf{b} \in \mathbb{R}^n, \mathbf{x}_0 \in \mathbb{R}^n, \varepsilon_r > 0$ Output: $\mathbf{x}_k \in \mathbb{R}^n$ with $\mathbf{Ax}_k \approx \mathbf{b}$ 1: $k = 0$ # JAC: $C = 2n^2/\text{it}$ GAU: $C = 2n^2/\text{it}$ 2: while $\ \mathbf{Ax}_k - \mathbf{b}\ _p > \varepsilon_r$ do # e.g. $\varepsilon_r = 10^{-3} \ \mathbf{b}\ _p$ 3: $k = k + 1$ # 上面给的 C 不含这步 4: Compute $\mathbf{y}_k = \mathbf{b} - \mathbf{N}\mathbf{x}_{k-1}$ 5: Solve $\mathbf{M}\mathbf{x}_k = \mathbf{y}_k$ # $\downarrow \mathbf{A} = \mathbf{D} + \mathbf{L} + \mathbf{U}$ 6: end while # JAC: $M = D, N = L + U$; GAU: $M = D + L, N = U$	

LSQ-QR LSQ via QR	$C = 2mn^2 - \frac{2}{3}n^3 + \mathcal{O}(m^2 + n^2 + mn)$
Input: $\mathbf{A} \in \mathbb{R}^{m \times n}, m \geq n, \text{rank}(\mathbf{A}) = n, \mathbf{b} \in \mathbb{R}^m$ $\uparrow$ by reduced QR Output: $\mathbf{x} \in \mathbb{R}^n$ with $\ \mathbf{Ax} - \mathbf{b}\ _2$ minimised 1: Compute reduced QR factorization $\mathbf{A} = \mathbf{Q}_1 \mathbf{R}_1$ # QR Algorithm 2: Compute $\mathbf{y} = \mathbf{Q}_1^\top \mathbf{b}$ # MV Algorithm # $C = 2mn$ 3: Solve $\mathbf{R}_1 \mathbf{x} = \mathbf{y}$ # BS Algorithm # $C = n^2$	

PI Power Iteration	$C = ?$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$ symmetric, $\mathbf{z}^{(0)} \in \mathbb{R}^n \setminus \{\mathbf{0}\}, \varepsilon > 0$ Output: $\mathbf{z}^{(k)} \in \mathbb{R}^n, \lambda^{(k)} \in \mathbb{R}$ s.t. $\mathbf{z}^{(k)} \approx \mathbf{x}_{\max}, \lambda^{(k)} \approx \lambda_{\max}$ 1: $k = 0$ 2: $\lambda^{(0)} = r_{\mathbf{A}}(\mathbf{z}^{(0)})$ 3: while $\ \mathbf{Az}^{(k)} - \lambda^{(k)}\mathbf{z}^{(k)}\ _2 > \varepsilon$ do # or: $ \lambda^{(k)} - \lambda^{(k-1)} > \varepsilon$ 4: $k = k + 1$ # or: $\ \mathbf{z}^{(k)} - \mathbf{z}^{(k-1)}\ _2 > \varepsilon$ 5: $\mathbf{z}^{(k)} = \mathbf{Az}^{(k-1)}$ 6: $\mathbf{z}^{(k)} = \frac{\mathbf{z}^{(k)}}{\ \mathbf{z}^{(k)}\ _2}$ # To improve stability and avoid large values 7: $\lambda^{(k)} = r_{\mathbf{A}}(\mathbf{z}^{(k)})$ # In fact: $\mathbf{z}^{(k)} = \frac{\mathbf{A}^k \mathbf{z}^{(0)}}{\ \mathbf{A}^k \mathbf{z}^{(0)} \ _2}$ and $\lambda^{(k)} = r_{\mathbf{A}}(\mathbf{z}^{(k)})$ 8: end while	

MV Matrix-vector multiplication	
$C = \sum_{i=1}^m 2n = 2nm = \mathcal{O}(mn)$	
Input: $\mathbf{A} \in \mathbb{R}^{m \times n}, \mathbf{x} \in \mathbb{R}^n$ Output: $\mathbf{y} = \mathbf{Ax}$ 1: for $i = 1, \dots, m$ do 2: $y_i = \mathbf{a}_i^\top \mathbf{x}$ # DP Algorithm 3: end for	

FS Forward substitution	
$C(n) = \sum_{j=1}^n [2(j-1) + 1] = n^2 = \mathcal{O}(n^2)$	
Input: $\mathbf{L} \in \mathbb{R}^{n \times n}$ (lower triangular), $\mathbf{b} \in \mathbb{R}^n$ Output: $\mathbf{y} \in \mathbb{R}^n$ such that $\mathbf{Ly} = \mathbf{b}$ 1: for $j = 1, \dots, n$ do 2: $y_j = \frac{1}{l_{jj}} \left(b_j - \sum_{k=1}^{j-1} l_{jk} y_k \right)$ 3: end for	

GEPP GE(partial pivoting)	$C(n) = \frac{2}{3}n^3 + 3n^2 - \frac{5}{3}n = \mathcal{O}(n^3)$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}, \mathbf{b} \in \mathbb{R}^n$ # numpy.linalg.solve use it Output: $\mathbf{x} \in \mathbb{R}^n$ satisfying $\mathbf{Ax} = \mathbf{b}$ 1: Find $\mathbf{PA} = \mathbf{LU}$ using Algorithm LUPP # LUPP Algorithm 2: Solve $\mathbf{Ly} = \mathbf{Pb}$ by forward substitution # FS Algorithm 3: Solve $\mathbf{Ux} = \mathbf{y}$ by back substitution # BS Algorithm	

LUCP LU factorisation with complete pivoting (not unique)	$C(n) = n^3 + n^2 - 2n$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$ Output: $\mathbf{L}, \mathbf{U}, \mathbf{P}, \mathbf{Q} \in \mathbb{R}^{n \times n}$ s.t. $\mathbf{PAQ} = \mathbf{LU}$ with $\mathbf{L}$ 单位下三角, $\mathbf{U}$ 可逆上三角 1: $\mathbf{U} = \mathbf{A}, \mathbf{L} = \mathbf{I}, \mathbf{P} = \mathbf{I}, \mathbf{Q} = \mathbf{I}$ 2: for $k = 1, \dots, n-1$ do 3: Choose $(i, j) \in \{(k, k), \dots, (n, n)\}$ which maximizes $ u_{ij} $ # Max Algorithm 4: Exchange row $(u_{k1}, \dots, u_{kn})$ with $(u_{ik}, \dots, u_{in})$ # 第 k 行和第 i 行的 $k \sim n$ 列交换 U 5: Exchange row $(l_{k1}, \dots, l_{k,k-1})$ with $(l_{i1}, \dots, l_{i,k-1})$ # 第 k 行和第 i 行的 $1 \sim k-1$ 列交换 L 6: Exchange row $(p_{k1}, \dots, p_{kn})$ with $(p_{i1}, \dots, p_{in})$ # 第 k 行和第 i 行的全部列交换 P 7: Exchange column $(u_{1k}, \dots, u_{nk})$ with $(u_{1j}, \dots, u_{nj})$ # 第 k 列和第 j 列的全部行交换 U 8: Exchange column $(q_{1k}, \dots, q_{nk})$ with $(q_{1j}, \dots, q_{nj})$ # 第 k 列和第 j 列的全部行交换 Q 9: for $m = k+1, \dots, n$ do 10: $l_{mk} = u_{mk}/u_{kk}$ 11: $(u_{mk}, \dots, u_{mn}) = (u_{mk}, \dots, u_{mn}) - l_{mk}(u_{kk}, \dots, u_{kn})$ 12: end for 13: end for	

(M)GS (Modified) Gram-Schmidt	$C(n) = 2n^3 + n^2 + n$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$, with columns $\mathbf{a}_1, \dots, \mathbf{a}_n$ Output: $\mathbf{Q} \in \mathbb{R}^{n \times n}$, with orthonormal columns $\mathbf{q}_1, \dots, \mathbf{q}_n$ $\mathbf{R} \in \mathbb{R}^{n \times n}$ non-singular upper triangular matrix; s.t. $\mathbf{A} = \mathbf{QR}$ 1: $\mathbf{R} = \mathbf{0}$ 2: for $j = 1, \dots, n$ do 3: $\mathbf{q}_j = \mathbf{a}_j$ 4: for $k = 1, \dots, j-1$ do 5: $r_{kj} = \mathbf{q}_k^\top \mathbf{a}_j$ $(r_{kj} = \mathbf{q}_k^\top \mathbf{q}_j)$ 6: $\mathbf{q}_j = \mathbf{q}_j - r_{kj} \mathbf{q}_k$ 7: end for 8: $r_{jj} = \ \mathbf{q}_j\ _2 = \sqrt{\mathbf{q}_j^\top \mathbf{q}_j}$ 9: $\mathbf{q}_j = \frac{\mathbf{q}_j}{r_{jj}}$ 10: end for	
OP Outer product	$C = \sum_{i=1}^n \sum_{j=1}^n 1 = n^2 = \mathcal{O}(n^2)$
Input: $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ Output: $\mathbf{A} = \mathbf{xy}^\top$ 1: for $i = 1, \dots, n$ do 2: for $j = 1, \dots, n$ do 3: $a_{ij} = x_i y_j$ 4: end for 5: end for	

LSQ-SVD LSQ via SVD	
Input: $\mathbf{A} \in \mathbb{R}^{m \times n}, m \geq n, \text{rank}(\mathbf{A}) = n, \mathbf{b} \in \mathbb{R}^m$ Output: $\mathbf{x} \in \mathbb{R}^n$ with $\ \mathbf{Ax} - \mathbf{b}\ _2$ minimised 1: Compute reduced SVD $\mathbf{A} = \mathbf{U}_1 \Sigma_1 \mathbf{V}^\top$ # see SVD note 2: Compute $\mathbf{z} = \mathbf{U}_1^\top \mathbf{b}$ # MV Algorithm # $C = 2mn$ 3: Solve $\Sigma_1 \mathbf{y} = \mathbf{z}$ # $C = n$ 4: Compute $\mathbf{x} = \mathbf{Vy}$ # MV Algorithm # $C = 2n^2$	

SII Shifted Inverse Iteration	$C = ?$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$ symmetric, $s \in \mathbb{R}, \mathbf{z}^{(0)} \in \mathbb{R}^n \setminus \{\mathbf{0}\}, \varepsilon > 0$ Output: $\mathbf{z}^{(k)} \in \mathbb{R}^n, \lambda^{(k)} \in \mathbb{R}$ s.t. $\mathbf{z}^{(k)} \approx \mathbf{x}_a, \lambda^{(k)} \approx \lambda_a$ near s 1: $k = 0$ 2: $\lambda^{(0)} = r_{\mathbf{A}}(\mathbf{z}^{(0)})$ # first $\mathcal{O}(n^3)$ cost; then $\mathcal{O}(n^2)$ cost/it 3: while $\ \mathbf{Az}^{(k)} - \lambda^{(k)}\mathbf{z}^{(k)}\ _2 > \varepsilon$ do 4: $k = k + 1$ 5: Solve $(\mathbf{A} - s\mathbf{I})\mathbf{y} = \mathbf{z}^{(k-1)}$ for $\mathbf{y}$ # via GEPP & Find LU only once 6: $\mathbf{z}^{(k)} = \frac{\mathbf{y}}{\ \mathbf{y}\ _2}$ 7: $\lambda^{(k)} = r_{\mathbf{A}}(\mathbf{z}^{(k)})$ # In fact: $\mathbf{z}^{(k)} = \frac{(\mathbf{A}-s\mathbf{I})^{-k} \mathbf{z}^{(0)}}{\ (\mathbf{A}-s\mathbf{I})^{-k} \mathbf{z}^{(0)}\ _2}$ and $\lambda^{(k)} = r_{\mathbf{A}}(\mathbf{z}^{(k)})$ 8: end while	

MM Matrix-matrix multiplication	
$C = \sum_{i=1}^m \sum_{j=1}^p 2n = 2mnp = \mathcal{O}(mnp)$	
Input: $\mathbf{A} \in \mathbb{R}^{m \times n}, \mathbf{B} \in \mathbb{R}^{n \times p}$ Output: $\mathbf{C} = \mathbf{AB}$ 1: for $i = 1, \dots, m$ do 2: for $j = 1, \dots, p$ do 3: $c_{ij} = \mathbf{a}_i^\top \mathbf{b}_j$ # DP Algorithm 4: end for 5: end for	

BS Back substitution	
$C(n) = \sum_{j=1}^n [2(n-j) + 1] = n^2 = \mathcal{O}(n^2)$	
Input: $\mathbf{U} \in \mathbb{R}^{n \times n}$ (upper triangular), $\mathbf{y} \in \mathbb{R}^n$ Output: $\mathbf{x} \in \mathbb{R}^n$ such that $\mathbf{Ux} = \mathbf{y}$ 1: for $j = n, \dots, 1$ do 2: $x_j = \frac{1}{u_{jj}} \left(y_j - \sum_{k=j+1}^n u_{jk} x_k \right)$ 3: end for	

GECP GE(complete pivoting)	$C(n) = n^3 + 3n^2 - 2n = \mathcal{O}(n^3)$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}, \mathbf{b} \in \mathbb{R}^n$ Output: $\mathbf{x} \in \mathbb{R}^n$ satisfying $\mathbf{Ax} = \mathbf{b}$ 1: Find $\mathbf{PAQ} = \mathbf{LU}$ using Algorithm LUCP # LUCP Algorithm 2: Solve $\mathbf{Ly} = \mathbf{Pb}$ by forward substitution # FS Algorithm 3: Solve $\mathbf{Uz} = \mathbf{y}$ by back substitution # BS Algorithm 4: Compute $\mathbf{x} = \mathbf{Qz}$	

QR Householder QR factorization	$C(n) = \frac{8}{3}n^3 + \mathcal{O}(n^2)$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$ Output: $\mathbf{Q} \in \mathbb{R}^{n \times n}$ orthogonal $\mathbf{R} \in \mathbb{R}^{n \times n}$ non-singular upper triangular matrix; s.t. $\mathbf{A} = \mathbf{QR}$ 1: $\mathbf{Q} = \mathbf{I}, \mathbf{R} = \mathbf{A}$ # 如果 $\mathbf{A} \in \mathbb{R}^{m \times n}, m \geq n$, loop 到 n , 且 3.6.7 的 n 改为 m 2: for $k = 1, \dots, n-1$ do 3: $\mathbf{u} = (r_{kk}, \dots, r_{nk}) \in \mathbb{R}^{n-k+1}$ 4: $\mathbf{v} = \mathbf{u} + \text{sign}(u_1) \ \mathbf{u}\ _2 \mathbf{e}_1$ # line.4-7 得到的 $\mathbf{Q}_k$: 5: $\mathbf{v} = \frac{\mathbf{v}}{\ \mathbf{v}\ _2}$ # 使得 $\mathbf{Q}_k \mathbf{u} = -\text{sign}(u_1) \ \mathbf{u}\ _2 \mathbf{e}_1$ 6: $\mathbf{H}_k = \mathbf{I} - 2\mathbf{w}\mathbf{w}^\top \in \mathbb{R}^{(n-k+1) \times (n-k+1)}$ # line.6-8 一起算, 见下 7: $\mathbf{Q}_k = \mathbf{I}_{k-1} \oplus \mathbf{H}_k \in \mathbb{R}^{n \times n}$ # ↓ line.6-8 $C = 4(n-k+1)^2 + (n-k+1)$ 8: $\mathbf{R} = \mathbf{Q}_k \mathbf{R}$ # 用矩阵分块, 只有右下角 $\mathbf{H}_k \mathbf{R}_{k:n, k:n}$ 要算 $\rightarrow$ 用 $\mathbf{H}_k$ 定义 Alg. MV+OP 9: $\mathbf{Q} = \mathbf{QQ}_k$ # If using in Alg. SQR , then $\mathbf{Q}' : \mathbf{y} = \mathbf{Q}_k \mathbf{y}$: $C = \frac{4}{3}n^3 + \mathcal{O}(n^2)$ (SQR/QR) 10: end for # $\mathbf{A} \in \mathbb{R}^{m \times n}$ reduced 的 $C = 2n^2m - \frac{2}{3}n^3 + \mathcal{O}(m^2 + n^2 + mn)$	
Max Finding index of maximum	$C(m) = m - 1$
Input: $\mathbf{z} \in \mathbb{R}^m$ Output: index i with $ z_i = \max_{j=1, \dots, m} z_j $ 1: $i = 1, \text{ max} = z_1 $ 2: for $j = 2, \dots, m$ do 3: if $ z_j > \text{max then}$ 4: $\text{max} = z_j $ 5: end if 6: end if 7: end for	

5 Appendix

5.1 Useful Theorems/Formulas

5.2 Others

Where the error come from calucation:

1. If your calculation involves 15 or more significant digits, you will likely encounter rounding errors. (Since $\varepsilon_m \approx 10^{-16}$ (relative accuracy), which is about 15 significant digits.)
2. **For GE Algorithm:** It is an unstable algorithm. e.g. $\mathbf{A} = [10^{-20} \ 1 \ \backslash \ 1 \ 1] \Rightarrow$ By **GE** Algorithm, $\hat{\mathbf{L}}\hat{\mathbf{U}} = \mathbf{A} + [0 \ 0 \ \backslash \ 0 \ -1]$ (error)
3. **MGS vs. GS:** In floating point arithmetic, the computed vectors $\{\hat{q}_1, \dots, \hat{q}_{j-1}\}$ will not be orthonormal. Algorithm MGS takes this into account when computing $\hat{q}_j$; Algorithm GS does not.